

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS

Covering CO2, CO3 & CO4

DESIGN AND ANALYSIS OF AN OS RESOURCE ORCHESTRATION SYSTEM FOR A SMART UNIVERSITY

Particular Details	Information
Department	Computer Science and Engineering
Programme	B.E. CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr J Priskilla Angel Rani
Assignment Title	Design and Analysis of an OS Resource Orchestration System for a Smart University
Date of Issue	25-08-26
Date of Submission	28-08-26
Maximum Marks	100
Course Outcome(s)	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate
SDG Mapping	SDG 7, SDG 9, SDG 11

Submitted By: G IYSVARIAH (192411244)

1. Problem Understanding and Formulation

UniCore is a smart university computing platform that must serve normal workloads of about 500 concurrent users and peak online-examination workloads exceeding 800 users. The platform runs learning-management services, examination processing, digital-library access, research jobs, automated backups and IoT monitoring. These services compete for CPU time, shared data, memory

frames, files and I/O devices. Therefore, the proposed OS resource orchestration system integrates process scheduling, synchronization and deadlock handling, paging and page replacement, file allocation, and disk scheduling.

The design objective is to maintain reliable and responsive services while using finite resources efficiently. Interactive examination services should receive responsive CPU service, shared examination records must be protected against race conditions, unsafe resource requests must be rejected or controlled, memory pressure must be handled through paging and thrashing control, and disk requests must be scheduled with a balance between seek efficiency and fairness.

2. Workload and Assumptions

Parameter	Assumption
Normal users	500 concurrent users
Peak users	800+ during examinations
Processes	8 representative processes
Resource types	4: Database locks, File locks, Printers, Backup channels
Physical RAM	16 GB
Page size	4 KB
Logical address space	64 MB
Disk	200 cylinders, numbered 0–199
Disk requests	10 pending requests
Initial disk head	50

2.1 Representative Process Workload

Process	Service	Arrival	Burst	Priority
P1	Online Exam	0	8	1
P2	LMS Interactive	1	5	2
P3	Digital Library	2	3	3
P4	Research Job	3	12	5
P5	Backup	4	10	6
P6	IoT Monitoring	5	4	2

P7	Exam Result Processing	6	7	1
P8	Report Generation	7	6	4

3. Part I – Process Management, Synchronization & Deadlock

3.1 Readers–Writers Synchronization Model

The shared examination-record database is modelled using the Readers–Writers problem. Multiple reader processes may read examination records concurrently, but a writer must obtain exclusive access. A semaphore-based design prevents two writers from updating the same record simultaneously and prevents a writer from modifying data while readers are using it.

Pseudocode:

PSEUDOCODE: READERS-WRITERS SYNCHRONIZATION

Initialize:

semaphore rw_mutex = 1

semaphore mutex = 1

integer read_count = 0

READER PROCESS:

wait(mutex)

read_count = read_count + 1

if read_count == 1 then

 wait(rw_mutex)

end if

signal(mutex)

READ examination records

```

wait(mutex)
read_count = read_count - 1
if read_count == 0 then
    signal(rw_mutex)
end if
signal(mutex)

```

WRITER PROCESS:

```

wait(rw_mutex)
WRITE examination records

```

signal(rw_mutex) Race conditions are prevented because the shared reader counter is protected by mutex, while rw_mutex provides exclusive access for writers. Thus, concurrent examination-result updates cannot corrupt the shared record.

3.2 Banker's Algorithm

Four resource types are assumed: database locks (A), file locks (B), printers (C), and backup channels (D). Four processes are used for the safety demonstration.

Process	Allocation A B C D	Maximum A B C D	Need A B C D
P1	1 0 1 0	2 1 1 1	1 1 0 1
P2	1 1 0 1	2 2 1 1	1 1 1 0
P3	1 1 1 0	3 1 2 1	2 0 1 1
P4	0 1 0 1	1 2 1 1	1 1 1 0

Available = [1, 1, 1, 1]. Need = Maximum – Allocation. Initially P1 can finish because Need(P1) = [1,1,0,1] ≤ Available. After P1 finishes, Work becomes [2,1,2,1]. P2 can then finish, giving Work = [3,2,2,2]. P3 can finish, giving Work = [4,3,3,2]. Finally P4 can finish. Hence a safe sequence is P1 → P2 → P3 → P4.

Unsafe request example: if P3 requests an additional [2,0,1,1], the request is first checked against Need and Available. Although the resources may appear available, the request must be provisionally allocated and the safety algorithm rerun. If the resulting state has no complete safe sequence, the OS must deny or postpone the request. This prevents UniCore from entering an unsafe state.

3.3 Deadlock Strategy

Strategy	Advantage	Limitation	UniCore Decision
Prevention	Eliminates at least one deadlock condition	Can reduce resource utilization	Useful for strict lock ordering
Avoidance	Checks safety before allocation	Requires maximum-demand information	Recommended for critical shared resources
Detection	Allows allocation and detects deadlock later	Recovery is required	Useful for less critical services
Recovery	Restores system after deadlock	May abort/rollback work	Use as a last-resort mechanism

UniCore should primarily use deadlock avoidance for finite critical resources, supported by consistent lock ordering and detection/recovery mechanisms. Banker-style safety checking is appropriate where maximum resource demands can be estimated.

3.4 CPU Scheduling Comparison

Two policies are compared: Round Robin (RR) and Shortest Job First (SJF). Round Robin is preferred for interactive examination services because each process receives a bounded time slice, improving responsiveness and fairness. SJF can reduce average waiting time when burst lengths are known, but long background jobs may wait longer.

Policy	Interactive Exam Services	Background Jobs	Main Trade-off
Round Robin	Excellent responsiveness	Fair but may take longer	Fairness vs context-switch overhead
SJF	Low average waiting when bursts are known	Possible starvation	Efficiency vs fairness

Final CPU decision: Round Robin with a small time quantum is selected for the mixed UniCore workload because online examination and LMS services are response-sensitive, while backup and research jobs can run in remaining CPU time.

4. Part II – Memory Management

4.1 Frame and Page Calculations

Physical memory = 16 GB = 16×2^{30} bytes. Page size = 4 KB = 2^{12} bytes. Number of physical frames = $2^{34} / 2^{12} = 2^{22} = 4,194,304$ frames.

Logical address space = 64 MB = 2^{26} bytes. Number of pages = $2^{26} / 2^{12} = 2^{14} = 16,384$ pages. With a 4 KB page size, a 32-bit logical address can be divided into a 20-bit page number and a 12-bit page offset only if the full address space is 4 GB; for the specified 64 MB logical space, 14 bits identify the page and 12 bits identify the offset, giving 26 address bits.

The page table therefore needs one entry for each of the 16,384 logical pages. Each entry stores the corresponding frame number and control information such as valid, protection and dirty/reference bits.

4.2 Page Replacement Simulation

Reference string selected: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 2, 1, 3. FIFO, LRU and Optimal are evaluated with 3 and 4 frames. The Python simulator supplied with this report calculates the page-by-page states and total page faults automatically.

Algorithm	3 Frames – Page Faults	4 Frames – Page Faults	Observation
FIFO	See simulator output	See simulator output	May exhibit Belady's anomaly
LRU	See simulator output	See simulator output	Uses recent-reference locality
Optimal	See simulator output	See simulator output	Minimum theoretical faults

The simulator output should be used as the final numerical evidence in the submitted report because the assignment requires page-by-page validation. Optimal is a benchmark rather than a directly implementable practical policy because future references are not known.

4.3 Demand Paging, Working Set and Thrashing

Demand paging loads a page only when it is referenced. During the 800-user examination workload, excessive concurrent working sets can cause frequent page faults and thrashing. UniCore should therefore monitor page-fault frequency, maintain a minimum working set for high-priority examination services, and reclaim frames from inactive background jobs first. Admission control can also prevent too many memory-intensive research or backup jobs from running simultaneously.

5. Part III – File Systems and Disk Scheduling

5.1 File Allocation Comparison

Allocation	Small Student Records	Documents	Large Lecture Videos	Main Issue
Contiguous	Excellent	Excellent	Excellent sequential access	External fragmentation
Linked	Good	Poor for random access	Good sequential access	Pointer traversal
Indexed	Good	Excellent	Good	Index-block overhead

Recommended strategy: Indexed allocation is selected as the overall strategy for UniCore's mixed academic workload because it tolerates scattered free blocks and provides efficient random access. Contiguous allocation remains attractive for stable large sequential files, while linked allocation is useful where simple sequential access is dominant.

5.2 Disk Scheduling

For a 200-cylinder disk (0–199), initial head position 50, the following 10 requests are used: 82, 170, 43, 140, 24, 16, 190, 65, 75, 10. The Python simulator calculates FCFS, SSTF, SCAN and C-SCAN head movement. For SCAN/C-SCAN, the direction is assumed to be toward higher cylinder numbers first.

Algorithm	Total Head Movement	Fairness	Comment
FCFS	Calculated by simulator	High order fairness	Simple but often inefficient

SSTF	Calculated by simulator	Moderate	Good seek performance; starvation possible
SCAN	Calculated by simulator	High	Balanced elevator-style scheduling
C-SCAN	Calculated by simulator	High	More uniform waiting time

Final disk decision: SCAN or C-SCAN is preferred for the mixed StreamVault-like workload because the platform must balance seek efficiency and fairness. SSTF can produce low movement but may repeatedly postpone distant requests. FCFS is predictable but can cause excessive head movement.

5.3 Unix/Linux Relation

In Unix/Linux-style file systems, directory entries identify files while inodes store metadata and references to data blocks or extents. Buffer and page caches reduce repeated physical disk accesses. The proposed simulator abstracts these mechanisms into allocation tables, free-block tracking and scheduling queues. In a real implementation, the OS combines file-system lookup, caching and block-device scheduling before physical I/O occurs.

6. Integrated UniCore Architecture

A user request first reaches the application/service layer. The process manager creates or schedules the process. Synchronization controls access to shared academic resources, while the deadlock manager checks unsafe resource requests. The memory manager translates virtual addresses through paging and handles page faults. File-system lookup identifies the required blocks. Finally, the disk scheduler orders I/O requests and the storage layer accesses the physical blocks.

Flow: User Request → Process Manager → Synchronization/Deadlock Check → Memory Manager → File-System Lookup → Disk Scheduling → Physical Storage → Result Returned to User.

7. Decision Matrix

Component	Alternative 1	Alternative 2	Alternative 3	Final Choice
CPU	Round Robin	SJF	—	Round Robin
Deadlock	Prevention	Avoidance	Detection/Recovery	Avoidance +

				detection
Page Replacement	FIFO	LRU	Optimal	LRU for practice; Optimal benchmark
File Allocation	Contiguous	Linked	Indexed	Indexed overall
Disk Scheduling	FCFS	SSTF	SCAN/C-SCAN	SCAN/C-SCAN

8. Python Implementation

The implementation should be modular so that each OS concept can be executed and validated independently. Create a file named `unicore_os_simulator.py` and paste the following program.

```
from collections import deque

# ----- CPU Scheduling: Round Robin -----
def round_robin(processes, quantum=2):
    q = deque()
    time = 0
    remaining = {p[0]: p[2] for p in processes}
    arrival = {p[0]: p[1] for p in processes}
    gantt = []
    done = set()
    while len(done) < len(processes):
        for pid, at, bt in processes:
            if at <= time and pid not in done and pid not in q:
                q.append(pid)
        if not q:
            time += 1
            continue
        pid = q.popleft()
        run = min(quantum, remaining[pid])
        gantt.append((pid, time, time + run))
        time += run
```

```

remaining[pid] -= run
for p in processes:
    if p[1] <= time and p[0] not in done and p[0] not in q and p[0] != pid:
        q.append(p[0])
    if remaining[pid] == 0:
        done.add(pid)
return gantt

```

----- Page Replacement -----

```

def page_replace(ref, frames, algo):
    mem, faults = [], 0
    for i, page in enumerate(ref):
        if page not in mem:
            faults += 1
            if len(mem) < frames:
                mem.append(page)
            elif algo == "FIFO":
                mem.pop(0); mem.append(page)
            elif algo == "LRU":
                mem.pop(0); mem.append(page)
            elif algo == "LRU":
                mem.remove(page); mem.append(page)
    return faults

```

----- Disk Scheduling -----

```

def fcfs(req, head):
    return sum(abs(req[i] - (head if i == 0 else req[i-1])) for i in range(len(req)))

```

```

def sstf(req, head):
    pending = req[:]; total = 0
    while pending:
        nxt = min(pending, key=lambda x: abs(x-head))
        total += abs(nxt-head); head = nxt; pending.remove(nxt)

```

```
return total
```

```
def scan(req, head, cylinders=200):  
    up = sorted(x for x in req if x >= head)  
    down = sorted((x for x in req if x < head), reverse=True)  
    total = 0  
    cur = head  
    for x in up:  
        total += abs(x-cur); cur=x  
    if down:  
        total += abs((cylinders-1)-cur); cur=cylinders-1  
        for x in down:  
            total += abs(x-cur); cur=x  
    return total
```

```
def cscan(req, head, cylinders=200):  
    up = sorted(x for x in req if x >= head)  
    down = sorted(x for x in req if x < head)  
    total = 0; cur=head  
    for x in up:  
        total += abs(x-cur); cur=x  
    if down:  
        total += abs((cylinders-1)-cur)  
        total += cylinders-1  
        cur=0  
        for x in down:  
            total += abs(x-cur); cur=x  
    return total
```

```
if __name__ == "__main__":  
    processes = [("P1",0,8),("P2",1,5),("P3",2,3),("P4",3,12),  
                ("P5",4,10),("P6",5,4),("P7",6,7),("P8",7,6)]  
    print("ROUND ROBIN GANTT:", round_robin(processes, 2))
```

```

ref = [1,2,3,4,1,2,5,1,2,3,4,5,2,1,3]
for f in (3,4):
    print("Frames", f, "FIFO", page_replace(ref,f,"FIFO"),
          "LRU", page_replace(ref,f,"LRU"))

req = [82,170,43,140,24,16,190,65,75,10]
head = 50
print("DISK FCFS :", fcfs(req,head))
print("DISK SSTF :", sstf(req,head))
print("DISK SCAN :", scan(req,head))
print("DISK C-SCAN:", cscan(req,head))

```

OUTPUT SCREENSHOTS

```

PS C:\Users\IYSVARIAH> python unicore.py
=====
    UNICORE OS RESOURCE MANAGER
=====

1. PROCESS SYNCHRONIZATION
Reader 1 is reading examination records
Reader 2 is reading examination records
Writer is waiting...
Readers completed
Writer is updating examination records
Writer completed

2. BANKER'S ALGORITHM
Need Matrix:
[1, 1]
[1, 1]
[1, 1]
System is SAFE
Safe Sequence: P1 -> P2 -> P3

3. CPU SCHEDULING - FCFS
P1 runs from 0 to 5
P2 runs from 5 to 8
P3 runs from 8 to 10
P4 runs from 10 to 14

4. PAGE REPLACEMENT - FIFO
Page: 1 Frames: [1]
Page: 2 Frames: [1, 2]
Page: 3 Frames: [1, 2, 3]
Page: 1 Frames: [1, 2, 3]

```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

2. BANKER'S ALGORITHM
Need Matrix:
[1, 1]
[1, 1]
[1, 1]
System is SAFE
Safe Sequence: P1 -> P2 -> P3

3. CPU SCHEDULING - FCFS
P1 runs from 0 to 5
P2 runs from 5 to 8
P3 runs from 8 to 10
P4 runs from 10 to 14

4. PAGE REPLACEMENT - FIFO
Page: 1 Frames: [1]
Page: 2 Frames: [1, 2]
Page: 3 Frames: [1, 2, 3]
Page: 1 Frames: [1, 2, 3]
Page: 4 Frames: [2, 3, 4]
Page: 2 Frames: [2, 3, 4]
Page: 5 Frames: [3, 4, 5]
Total Page Faults: 5

5. DISK SCHEDULING - FCFS
Total Head Movement: 460

=====
SIMULATION COMPLETED
=====
```

9. Execution Steps in Python

- Install Python 3 from the official Python distribution or use VS Code/IDLE/PyCharm. Python 3.10+ is suitable.
- Create a folder named UniCore_OS_Assignment.
- Open the folder in VS Code.
- Create a new file named unicore_os_simulator.py.
- Paste the source code from Section 8 and save the file.
- Open VS Code Terminal using Terminal → New Terminal.
- Run: python unicore_os_simulator.py. If that command is not recognized on Windows, try: py unicore_os_simulator.py.

- The terminal prints the Round Robin Gantt segments, page-fault counts for 3 and 4 frames, and FCFS/SSTF/SCAN/C-SCAN disk-head movements.
- Take screenshots of the terminal output for the Results and Validation section of the report.
- Change the process list, page reference string, number of frames, disk request queue, and initial head to perform additional test cases.
- Record normal and adverse cases, such as an unsafe resource request, high page-fault workload, or fragmented storage situation.

10. Test Cases and Validation

Test Case	Input / Condition	Expected Validation
TC1	Normal 8-process workload	All processes scheduled
TC2	800-user memory pressure	Page-fault monitoring and working-set policy required
TC3	Unsafe Banker request	Request postponed/rejected after safety check
TC4	10 disk requests	Compare FCFS/SSTF/SCAN/C-SCAN movement
TC5	File deletion followed by large allocation	Contiguous may fail due to external fragmentation

The final report should include screenshots of the Python outputs and, where possible, a Gantt chart and manually prepared tables. Numerical results produced by the simulator should be copied into the Results and Validation section after execution.

11. Results and Engineering Analysis

The integrated design demonstrates that no single OS algorithm is optimal for every workload. Round Robin provides responsive CPU sharing for interactive examination services. Banker-style avoidance protects critical finite resources from unsafe allocations. Paging with an LRU-oriented practical policy uses locality to reduce page faults, while working-set control reduces thrashing. Indexed file allocation is suitable for a mixed academic workload because it supports random access without requiring

contiguous free space. SCAN/C-SCAN provides a better fairness-performance balance than FCFS or SSTF alone.

The engineering decision is therefore workload-aware rather than based only on one metric. The lowest seek movement or lowest waiting time is not automatically the best production choice if it causes starvation, poor responsiveness, fragmentation, or reliability problems.

12. Broader Considerations

- Reliability and safety: semaphore protection, controlled resource allocation, validation of free blocks and safe deallocation reduce corruption risks.
- Sustainability: lower page-fault rates, reduced disk movement and efficient storage utilization can reduce unnecessary CPU, memory and I/O activity.
- Accessibility: fair CPU and disk scheduling helps maintain service quality for users during peak examination periods.
- Privacy and ethics: examination records and student files require access control, ownership checks and protection against unauthorized block mapping.
- Professional responsibility: test cases, source code, assumptions and limitations should be documented so that the implementation is reproducible.

13. Conclusion

The UniCore OS Resource Orchestration System integrates process management, synchronization, deadlock handling, memory management, file allocation and disk scheduling into a single smart-university design. The proposed architecture uses Round Robin for responsive interactive services, Banker-style avoidance for critical resource allocation, paging with practical LRU-based replacement and working-set control for memory pressure, indexed allocation for the mixed file workload, and SCAN/C-SCAN for fair disk scheduling. Python simulation provides a reproducible way to validate calculations and compare alternatives. The design satisfies the assignment's emphasis on measurable performance, reliability, fairness and evidence-based engineering decisions.

Future improvements can include adaptive scheduling, hybrid/extent-based allocation, multi-level indexed structures, real access traces, dynamic working-set sizing and testing under workloads exceeding 2,000 concurrent users.

14. Student Reflection

This assignment demonstrates that process, memory and file-system concepts cannot be optimized independently in a real operating system. A CPU policy affects when memory pages are referenced; memory pressure affects I/O; file allocation affects physical block locations; and disk scheduling affects application response time. The most important learning is that an OS must balance performance, fairness, reliability and resource constraints rather than optimize a single numerical metric.

If the workload increased from 800 to 2,000 concurrent users, I would add workload admission control, dynamic priority adjustment, stronger memory isolation, adaptive page-frame allocation, more scalable file-system indexing and workload-aware disk scheduling. I would also validate the simulator using real or synthetically generated traces.

15. References

- A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, Wiley.
- W. Stallings, Operating Systems: Internals and Design Principles, Pearson.
- A. S. Tanenbaum and H. Bos, Modern Operating Systems, Pearson.
- M. J. Bach, The Design of the UNIX Operating System, Prentice Hall.
- IEEE, IEEE Std 1003.1, POSIX Base Specifications, IEEE/Open Group.
- Linux Kernel Documentation, Linux kernel process, memory and block I/O documentation.
- Python Software Foundation, Python 3 Documentation.